Diagnose packet loss

Applies to: Supported versions of Windows Client

Packet loss occurs whenever a network packet doesn't reach its intended destination. Some packet loss is normal, and doesn't always cause higher-level networking issues. At other times, packet loss can reduce performance, and cause application programming interfaces (APIs) or applications to fail.

Capture and diagnose packet loss

The first step when conducting a packet loss investigation is to capture Packet Monitor (Pktmon) traces, typically by using the pktmon command-line workflow. Pktmon can capture packet traces, attribute local packet loss to specific reasons and code locations, and collect packet loss statistics. When combined with Wireshark analysis of protocol-level behavior, pktmon traces are sufficient to identify the root causes of many cases of packet loss.

If pktmon diagnostics are inconclusive, more exhaustive component-level traces can be captured using the `netsh.exe trace start scenario=InternetClient` or `netsh.exe trace start scenario=InternetServer` command for client and server scenarios respectively. After the events are captured, stop the trace by using the `netsh.exe trace stop` command. These component-level traces are noisy and not clear, but often contain additional context before or after a local packet is dropped. For remote loss, they might indicate that the local system infers the occurrence of packet loss. The traces can be converted to text using `netsh.exe trace convert nettrace.etl`, opened in Windows Performance Analyzer, or used with any other Event Tracing for Windows (ETW) tool.

If the network interface card (NIC) is suspected as a cause of the packet loss, you can monitor its discard counters through any performance counters interface or the Get-NetAdapterStatistics cmdlet.

Common causes of local packet loss

Local packet loss is fully observable and can be caused by various internal and external factors.

- Local policy

  Inspection software might cause packets from remote machines to be dropped by default, for example, when the Windows Firewall rejects inbound connection attempts. Cybersecurity or anti-malware software on the system can also cause these issues.

- Low resources

  If the system or socket runs out of resources to handle the packet, the packet is dropped. Examples of resource limits include physical memory on the system and socket send or receive buffers. Depending on the resource limit, these events might last only microseconds, for example, when the system's CPU can't react quickly enough to a full receive buffer.

- ARP or ND failure

  If the next hop for an outbound packet doesn't respond to Address Resolution Protocol (ARP) or neighbor discovery (ND) requests, then packets sent to that next hop are dropped on the local system. Packets might also be dropped during ARP or ND processes if the ARP or ND packet queue limit is exceeded. The ARP or ND packets themselves are typically not dropped locally and belong to the remote packet loss category.

- No route

If the network layer can't find a valid route to the destination, packets might be dropped.

- Invalid packet

  If the packet headers are invalid, the packet might be dropped. For example, the packet headers contain an invalid checksum or field value.

Common causes of remote packet loss

Remote packet loss isn't directly observable to the local machine when the packet is dropped. Internet Protocol (IP) and most layers below it are "best effort" and not reliable. The end-to-end principle requires endpoints to implement reliability within their protocols if resilience to packet loss is required. In some scenarios, the network or remote endpoint sends a protocol-specific error message indicating the reason for the loss. However, in many cases, the only indication of packet loss is a lack of response.

- Congestion

  Loss-based congestion control algorithms send faster and faster until a packet is lost. If the algorithm infers the loss is caused by congestion, it temporarily reduces the rate of sending in response. These algorithms require a small amount of loss to provide a feedback signal.

- Remote policy

  The network or remote machine might drop packets according to its own policy.

- Destination unreachable

  This can occur if the remote machine doesn't have a socket bound to the remote port, remote machine is offline, or the network can't find a path to the remote machine.

- Session loss

  If the network (including stateful Network Address Translation (NAT), firewalls, and load balancers) or the remote machine is reset or doesn't receive a packet recently, its session context might expire, and subsequent packets are dropped.

- Maximum Transmission Unit (MTU) drops

  If the size of the packet exceeds the maximum transmission size of a network link along the path to the remote machine, MTU drops might produce an error: Internet Control Message Protocol (ICMP) fragmentation required or packet too big.

Example of Packet Monitor traces

Run the following commands:

Console

```
pktmon.exe start -c
pktmon.exe stop
pktmon.exe etl2txt PktMon.etl
```

The resulting **PktMon.txt** file contains lines that resemble the following:

Output

```
[30]0000.0000::<DateTime> [Microsoft-Windows-PktMon] Drop: PktGroupId
8444249301423149, PktNumber 1, Appearance 0, Direction Rx , Type IP , Component 49,
Filter 1, DropReason INET: transport endpoint was not found , DropLocation 0xE000460A,
OriginalSize 402, LoggedSize 148
       Drop: ip: 192.168.5.88.50005 > 192.168.5.68.50005: UDP, length 374
```

This information indicates the inbound UDP packet destined to port 50005 is dropped because no local socket is bound to the port.

Example of Network Shell traces

Run the following commands:

Console

```
netsh.exe trace start scenario=InternetClient
netsh.exe trace stop
netsh.exe trace convert NetTrace.etl
```

The resulting **NetTrace.txt** file contains lines that resemble the following:

Output

```
[30]0000.0000::<DateTime> [Microsoft-Windows-TCPIP]TCPIP: Network layer (Protocol
1(ICMP), AddressFamily = 2(IPV4)) dropped 1 packet(s) on interface 13. SourceAddress =
192.168.5.68. DestAddress = 192.168.5.88. Reason = 9(Inspection drop). Direction =
0(Send). NBL = 0xFFFFE189BEAF3AC0.
```

This information indicates the outbound ICMP packet is dropped due to Windows Filtering Platform (WFP) inspection. The next step for WFP is to follow the WFP live drops troubleshooting steps.

In another scenario, a previously sent TCP segment isn't acknowledged by the remote endpoint, and eventually a local retransmit timer fires, causing TCP to resend some of the potentially lost data:

Output

```
[31]0000.0000::<DateTime> [Microsoft-Windows-TCPIP]TCP: Connection 0xFFFFE189BD811AA0
0(RetransmitTimer) timer has expired.
[31]0000.0000::<DateTime> [Microsoft-Windows-TCPIP]TCP: Tail Loss Probe Event
Connection = 0xFFFFE189BD811AA0, Event = 2(TimerFired).
[31]0000.0000::<DateTime> [Microsoft-Windows-TCPIP]TCP: Tail Loss Probe Send
Connection = 0xFFFFE189BD811AA0 SndUna = 2526318360, SndMax = 2526321759,
SendAvailable = 3399, TailProbeSeq = 2526320299, TailProbeLast = 2526321759,
ControlsToSend = 0, ThFlags = 16.
[31]0000.0000::<DateTime> [Microsoft-Windows-TCPIP]TCP: connection 0xFFFFE189BD811AA0
(local=192.168.5.68:55330 remote=6.6.0.27:443): TCP send event, SeqNo = 2526320299,
BytesSent = 1460, CWnd = 18538, SndWnd = 197632, SRtt = 17631, RttVar = 4947, RTO =
300, RcvWnd = 65535, PacingRate = 0, State = 4(EstablishedState), CongestionState = 0,
SndUna = 2526318360, SndMax = 2526321759, RecoveryMax = 0, RcvBufSet = 0(FALSE),
MaxRcvBuf = 65535.
[31]0000.0000::<DateTime> [Microsoft-Windows-TCPIP]TCP: connection =
0xFFFFE189BD811AA0 send tracker marked a transmit as rexmit. Start = 2526320299, End =
2526321759, Timestamp = 467744252, InFlightCount = 2, SackedBytes = 0, BytesInFlight =
4859.
[31]0000.0000::<DateTime> [Microsoft-Windows-TCPIP]TCP: Connection 0xFFFFE189BD811AA0
0(RetransmitTimer) timer started. Scheduled to expire in 300 ms. Processor 31:
LastInterruptTime 305324952689 100-ns ticks; LastMicrosecondCount 30532515324 msec
```

More information

- Troubleshoot TCP/IP connectivity
- How to capture a NetLog dump
- How to collect a network trace
- Inspect network activity

Last updated on 02/12/2026